

Reviewer Pack — Minimal Rank of Motivic Sheaves Bounds Disjointness Communic...

Entry bb896e5b344d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 22:19:34 UTC

Minimal Rank of Motivic Sheaves Bounds Disjointness Communication Complexity

Entry ID: bb896e5b344d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 22:19:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Motivic Sheaf Theory) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any n -ary Boolean function f , the minimal rank of its associated motivic sheaf S_f is a lower bound on the randomized communication complexity $CC_R(f)$.’, ‘Specifically, if S_f has rank r , then $CC_R(f) \geq \Omega(r^2 \log n)$.’, ‘Additionally, there exists a constructive mapping from instances of disjointness to motivic sheaves such that the minimal rank of the associated sheaf matches the disjointness complexity.’]

Rationale (proposer’s reasoning):

[‘Motivic sheaves provide a powerful tool for studying algebraic properties of spaces, and their ranks have been used in various contexts. This conjecture leverages this concept to potentially uncover new insights into communication complexity.’, ‘The motivic rank of an object is known to capture complex arithmetic information, which may be related to the difficulty of computations that involve communication.’, ‘This bridge might expose a previously unexplored connection between algebraic geometry and communication complexity, leading to new algorithmic techniques or theory.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 64f265b4b4190143

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the ratio of the minimal rank to the randomized communication complexity across all seeds is greater than or equal to 1 and no seed produces a ratio less than 0.9, indicating a strong correlation between the two metrics.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "motivic sheaf theory" AND "communication complexity" AND disjointness - "minimal rank" AND "motivic sheaves" AND CC_R - "disjointness complexity" mapping TO "motivic sheaves"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_subtract(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def identity_matrix(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]
```

```

def matrix_inverse(A):
    n = len(A)
    I = identity_matrix(n)
    A_augmented = [A[i] + I[i] for i in range(n)]

    # Gaussian elimination with partial pivoting
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        if A[max_row][i] == 0:
            raise ValueError("Matrix is not invertible")

        A_augmented[i], A_augmented[max_row] = A_augmented[max_row], A_augmented[i]

        for j in range(n):
            A_augmented[i][j] /= A_augmented[i][i]

        for k in range(n):
            if k != i:
                factor = A_augmented[k][i]
                for j in range(2*n):
                    A_augmented[k][j] -= factor * A_augmented[i][j]

    return [row[n:] for row in A_augmented]

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    elif n == 2:
        return A[0][0]*A[1][1] - A[0][1]*A[1][0]
    else:
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1)**j * A[0][j] * determinant(submatrix)
        return det

def rank(A):
    n, m = len(A), len(A[0])
    if n == 0 or m == 0:
        return 0

    # Gaussian elimination to find the rank
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        if A[max_row][i] == 0:
            continue

        A[i], A[max_row] = A[max_row], A[i]

        for j in range(m):
            A[i][j] /= A[i][i]

        for k in range(n):
            if k != i:
                factor = A[k][i]

```

```

        for j in range(m):
            A[k][j] -= factor * A[i][j]

    return sum(1 for row in A if any(row))

def grothendieck_group_rank(clauses):
    n = len(clauses)
    I = [[0]*n for _ in range(n)]
    for i, clause in enumerate(clauses):
        for j in range(i+1, n):
            if set(clause).isdisjoint(set(clauses[j])):
                I[i][j] = 1
                I[j][i] = 1

    return rank(I)

def disjointness_complexity(n):
    # Simulate a random instance of the disjointness problem
    A = [random.choice([0, 1]) for _ in range(2**n)]
    B = [random.choice([0, 1]) for _ in range(2**n)]

    return sum(A[i] != B[i] for i in range(2**n))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            clauses = [[random.randint(0, n-1) for _ in range(random.randint(1, 3))] for _ in range(n)]
            rank_value = grothendieck_group_rank(clauses)
            cc_value = disjointness_complexity(n)

            if rank_value == 0:
                continue

            ratio = Fraction(rank_value**2 * math.log(n), cc_value)
            total_metric_value += ratio
            instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = mean_metric_value >= 1 and all(ratio >= 0.9 for ratio in [Fraction(rank**2 * math.log(n), cc_value) for n in n_values])
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Ratio of Rank^2 * log(n) to CC_R",
        "metric_value": float(mean_metric_value),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}}")
    results.append(result)

mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8bd022b9.py", line 161, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8bd022b9.py", line 139, in run_trial
    ratio = Fraction(rank_value**2 * math.log(n), cc_value)
              ~~~~~^~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12059
2	preregistration	ollama_remote	glm4:latest	0	0	5420
3	novelty	ollama_remote	glm4:latest	0	0	4632
4	novelty	ollama_remote	glm4:latest	0	0	5999
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15178
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9401
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13305
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19227
9	judge	ollama_remote	glm4:latest	0	0	7959

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93178 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bb896e5b344d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bb896e5b344d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bb896e5b344d.tar.gz` (if generated)